

**UNIVERSITÀ DEGLI STUDI DI CASSINO E
DEL LAZIO MERIDIONALE**

Dipartimento di Ingegneria Elettrica e dell'Informazione

Tesi di Laurea
in
Ingegneria Informatica e delle Telecomunicazioni

**Architettura Software Modulare
per Ponti Radio ad Alta Efficienza
con Algoritmi di Ritrasmissione e Correzione**

Relatore:

Prof. Mario Molinara

Candidato:

Andrea Roccaccino
Mat. 61336

Correlatore:

Prof. Ciro D'Elia
Prof. Francesco Porpora

Anno Accademico 2025/2026

*Un ringraziamento particolare va
al **Prof. Mario Molinara** per il costante confronto tecnico ed editoriale
al **Dott. Pavel Khlebovich** per il supporto alla ricerca (R&D)
a **Lucas Capuano** per il suo contributo nello sviluppo dell'hardware.
Alla mia compagna, per aver alleggerito ogni passo di questo percorso.
Alla mia famiglia, per aver creduto in me
e per avermi supportato in ogni momento.*

Indice

Introduzione	ix
1 Analisi preliminare	1
1.1 Soluzioni disponibili	1
1.2 Obiettivi e Requisiti	2
1.2.1 Inquadramento del problema	
1.2.2 Astrazione: la pila OSI	
1.2.3 Requisiti tecnici	
2 Strutture base del Software	5
2.1 Struttura a catena	5
2.2 Il blocco Sorgente	7
2.3 Il blocco Radio	8
2.3.1 Correzione degli errori del livello fisico	
2.3.2 Test dei bit flip	
2.3.3 Adattatore USB	
2.4 Il blocco Pacchettizzatore	9
2.4.1 Funzionamento del Pacchettizzatore	
2.5 Blocchi aggiuntivi: Cifratura e Compressione	10
2.6 Ottimizzazione del software	10
3 Segmentazione e HARQ	11
3.1 La segmentazione	11
3.1.1 Messaggi dalle dimensioni sconosciute	
3.1.2 Modalità di segmentazione	
3.2 Algoritmo ARQ	12
3.2.1 Modalità di funzionamento	
3.2.2 Formato dei pacchetti	
3.2.3 Interno dell'Header	
3.3 Flusso di lavoro con disturbi	16
4 Gestione multithreaded dei dati	17
4.1 Considerazioni sulle strutture disponibili	17
4.2 Soluzione implementata	18
4.3 Soluzione della terza versione	18
4.4 Gestione dell'ambiente multithreaded	19
4.4.1 Variabile Atomic	
4.4.2 Mutex e Unique Lock	
4.4.3 Le variabili condizione	

5	Sistema di configurazione utente	21
5.0.1	Configurazioni a Compile time e Runtime	
5.0.2	Formato del file di preset	
5.1	Struttura della classe Impostazioni	22
5.2	Interfaccia di configurazione	23
6	Il Circuito Stampato	25
6.1	Requisiti del Banco di Prova	25
6.2	Realizzazione del PCB	25
6.2.1	Circuiti Stampati Sviluppati	
6.2.2	Utilizzo in Scuderia UNICAS	
6.3	Prove sul campo	28
6.3.1	Svolgimento della prova e risultati	
6.3.2	Ricezione negli uffici	
6.3.3	Ipotesi di copertura in autodromo	
7	Installazione del Software	29
7.1	Piattaforma per l'implementazione	29
7.2	Preparazione di Raspberry Pi e CMake	29
7.3	Il file CMakeLists.txt	30
8	Conclusioni	31
8.1	Valutazione del software	31
8.1.1	Valutazione qualitativa	
8.1.2	Valutazione quantitativa	
8.1.3	Valutazione overhead	
8.2	Interpretazione dei risultati	33
	Glossario	37

Elenco degli snippet

2.1	L'interfaccia dedicata allo scambio di messaggi nella catena.	5
2.2	Classe messaggio.	6
2.3	Divisione messaggi in base al ruolo.	6
2.4	Generico elemento della catena.	7
2.5	Funzioni da implementare in un pacchettizzatore.	10
4.1	Enumerazione degli eventi poll.	18
4.2	Esempio di mutex con unique lock.	19
4.3	Esempio di condition variable.	20
5.1	Esempio classe impostazioni.	22
5.2	Esempio classe impostazioni equivalente.	22
5.3	Interfaccia oggetto per le impostazioni.	23
5.4	Modalità di applicazione delle impostazioni per un oggetto.	23

Elenco delle figure

1.1	Triangolo dei vincoli	1
1.2	Pila OSI e Implementazione corrispondente	3
2.1	Blocchi fondamentali del livello 3 della pila OSI	5
2.2	Pipeline di esempio	5
2.3	Flusso dati del Pacchettizzatore	9
3.1	Segmentazione del frame.	11
3.2	Struttura di un pacchetto HARQ	14
3.3	Suddivisione del primo byte di Header	15
3.4	Flusso di comunicazione ARQ con disturbi	16
6.2	Strati PCB per nRF24L01	27
6.3	Render PCB RX	27
6.4	Ipotesi di copertura in autodromo	28

Introduzione

La presente trattazione si incentra sullo sviluppo di un software in C++ per una coppia di computer con Debian (*Raspberry Pi*) come minimo requisito hardware e software, abbinati a due coppie di moduli radio *nRF24L01+*.

Risoluzione del problema

L'obiettivo del presente lavoro è consentire lo scambio di messaggi generici attraverso un supporto trasmissivo arbitrario, i cui criteri di selezione del mezzo e modalità di utilizzo possono variare nel tempo.

Il sistema deve garantire tempestività e integrità, risultando trasparente all'utente finale rispetto ai meccanismi di trasferimento sottostanti. Ulteriori requisiti tecnici sono definiti al paragrafo 1.2.3.

Un primo approccio naïf implementa sicuramente la tecnica del *character stuffing*: ci si affida alla correzione degli errori del livello superiore presumendo che tutti i byte siano ricevuti correttamente. Il risultato è un flusso di dati facilmente desincronizzabile.

Si introduce quindi una famiglia di algoritmi più robusta: l'*Automatic Repeat Request* (ARQ).

Riferimenti del lavoro

Questa trattazione si riferisce alla seconda versione del software [1]. Eventuali divergenze con la terza versione sono segnalate. L'algoritmo ARQ descritto nel capitolo 3 rimane invariato.

Capitolo 1

Analisi preliminare

1.1 Soluzioni disponibili

Al momento della stesura, non esiste una singola tecnologia di comunicazione adatta a tutte le esigenze, nonostante le caratteristiche della tecnologia 5G.

È sempre opportuno e necessario confrontare le tecnologie disponibili con i propri requisiti.

Il mercato delle telecomunicazioni è suddiviso principalmente in due categorie:

- Reti mobili e WiFi, altre soluzioni punto-punto a media e lunga distanza. Offrono alta velocità e latenza ridotta, ma comportano costi operativi non trascurabili e dipendenza da infrastrutture terze.
Nei casi in cui la copertura dovesse risultare insufficiente o indisponibile è richiesto l'intervento del gestore dell'infrastruttura, una criticità significativa per entrambi gli attori della rete.
- Moduli Embedded (LoRa, nRF24, HC-06). Flessibili ed economici, offrono una copertura variabile ma bassa velocità di scambio con transazioni di piccole dimensioni.
Richiedono lo sviluppo di firmware dedicato, spesso in C++.

Risulta agevole individuare la soluzione ottimale per minimizzare i costi: determinante è la capacità di pianificare e controllare l'ambiente di scambio dati.

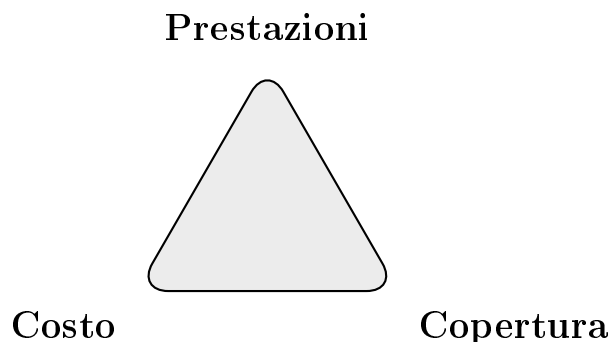


Figura 1.1: Triangolo dei vincoli - i vertici non sono massimizzabili contemporaneamente

1.2 Obiettivi e Requisiti

Esiste un divario significativo tra le prestazioni garantite dai gestori e la flessibilità dei moduli embedded. Come mostrato in Figura 1.1 risulta impossibile massimizzare simultaneamente copertura, prestazioni e contenimento dei costi.

In seguito a questa constatazione è naturale chiedersi se esistano soluzioni intermedie capaci di bilanciare tali vincoli. Sebbene a livello hardware non vi siano standard diffusi, è possibile mitigare i limiti fisici tramite il software. Progetti come *LoRaWAN* e *RF24Ethernet* operano in tale direzione per moduli LoRa e serie nRF24L01/nRF52x. Tuttavia l'evoluzione dell'hardware può richiedere delle reimplementazioni onerose, rallentando l'adozione delle nuove soluzioni.

1.2.1 Inquadramento del problema

La vastità del mercato e la variabilità dei requisiti tecnici rendono complessa l'individuazione di una soluzione ottimale basata sull'hardware. Frequentemente la scelta ricade sul modulo radio più ragionevole a cui poi abbinare una soluzione software di carattere *best-effort*; il suo riutilizzo in altri contesti necessita di adattamenti con costi aggiuntivi non previsti.

Il problema è chiaro ed anche una sua possibile soluzione: risulta estremamente utile un software capace di risolvere il problema della trasmissione in modo efficace, prescindendo dal modulo fisico sottostante. La scelta del dispositivo radio e conseguente riadattamento hanno un impatto estremamente ridotto nel processo di progettazione e sviluppo, poiché un intero livello di responsabilità è stato rimosso tramite astrazione.

1.2.2 Astrazione: la pila OSI

Il problema della comunicazione è intenzionalmente **impostato in modo tale da non conoscere a priori i dati da scambiare**.

Solitamente tutti i dettagli del progetto sono conosciuti, inclusi i dati da trasmettere. Questa conoscenza avvantaggia la scrittura della soluzione best-effort, più rapida da implementare rispetto a quella proposta in questo lavoro, ma non solo comporta lo svantaggio di non essere riutilizzabile: in alcuni casi si pone un problema di compatibilità se la sorgente dati è incompatibile con l'ambiente.

Esempio: un flusso video H264 inviato in rete locale deve essere catturato e riadattato per la trasmissione nel sistema, incidendo negativamente sulle limitate risorse computazionali. Oppure se il flusso è assolutamente incompatibile, può essere necessaria una implementazione personalizzata, estremamente costosa.

Si modella un sistema composto da due entità dove una riceve un flusso continuo di pacchetti e l'altra lo eroga.

La pila OSI in figura 1.2 delinea perfettamente i punti dove è opportuno operare.

È necessario un programma, definito *driver* o *middleware*, in grado di accettare un generico flusso dati e gestirlo totalmente in relazione al modulo radio scelto.

L'instradamento nei livelli inferiori al terzo è gestito dal modulo radio, mentre nei livelli superiori non è necessario poiché le casistiche possibili non lo richiedono, ovvero:

Applicazione Personalizzata Se l'applicazione è personalizzata (come un programma per Arduino) allora sarà l'unico mittente e destinatario dei dati.

Flusso Se il collegamento è terminato su un flusso dati (rete locale, bus) l'instradamento è già implementato nello stesso livello della sorgente.

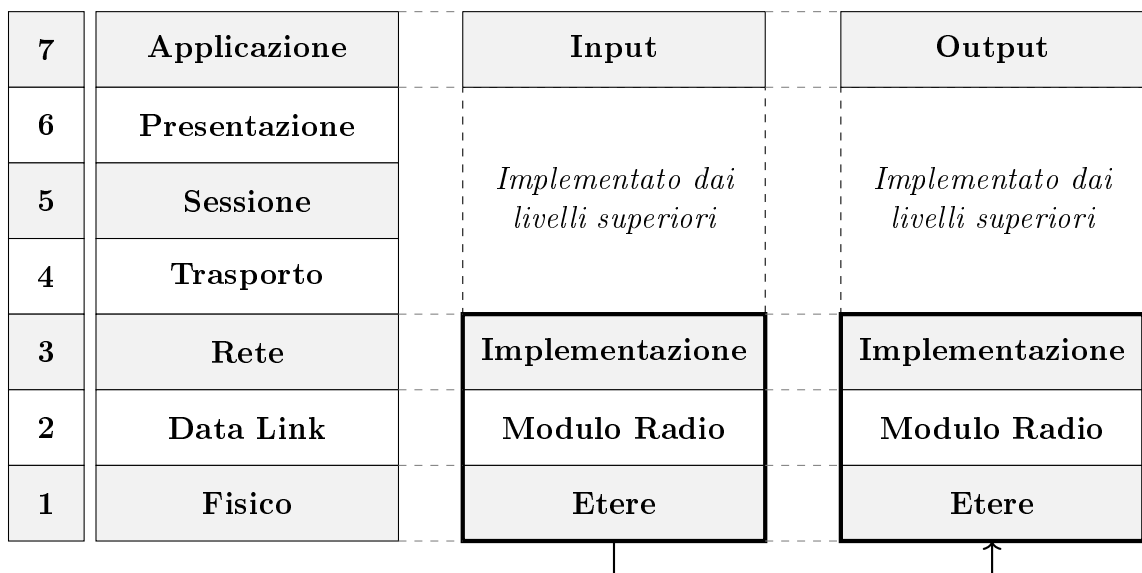


Figura 1.2: Pila OSI e Implementazione corrispondente

1.2.3 Requisiti tecnici

Al fine di provare il programma durante lo sviluppo è stato scelto un hardware di riferimento a partire dai seguenti requisiti:

Copertura minima	Raggio di 1 km
Flusso dati desiderabile	Controllo remoto, telemetria e streaming video in tempo reale
Costo	Idealmente sotto €10 cad.
Comunicazione	Full-Duplex

Tabella 1.1: Requisiti per il sistema

Il modulo radio più vicino a queste caratteristiche, seppur Half-Duplex, è *nRF24L01+*: un dispositivo prodotto da Nordic Semiconductor. Dispone di un protocollo proprietario (*ShockBurst* [2]) in grado di scambiare messaggi in una sola transazione con una velocità di trasmissione selezionabile tra 250kbps, 1Mbps, 2Mbps [2], copertura massima di 1km, costo tra €1 e €5 cad.

Da abbinare a una coppia di Raspberry Pi: un Single Board Computer con doppio bus SPI in grado di comunicare con due moduli nRF24L01+ simultaneamente.

Quest'ultimo punto è rilevante poiché parallelizzare due collegamenti Half-Duplex in verso opposto risulta in un collegamento Full-Duplex.

Nordic Semiconductor. *nRF24L01: Single Chip 2.4GHz Transceiver (English)*. URL: https://cdn.sparkfun.com/datasheets/Wireless/Nordic/nRF24L01_Product_Specification_v2_0.pdf (visitato il giorno 12/12/2025)

Capitolo 2

Strutture base del Software

Questo capitolo affronta le basi necessarie alla scrittura del software per il ponte radio, seguendo le *best practices* della programmazione a oggetti.

2.1 Struttura a catena

Dall'introduzione si può dedurre che la struttura del software deve avere un minimo di tre blocchi per essere funzionale: sorgente, pacchettizzatore, radio.



Figura 2.1: Blocchi fondamentali del livello 3 della pila OSI

L'implementazione di nuovi blocchi nella catena è semplice: supponendo di introdurre compressione e crittografia dei frame e correzione degli errori di canale, la catena completa risulterebbe come in figura 2.2 senza alterare il funzionamento originale.

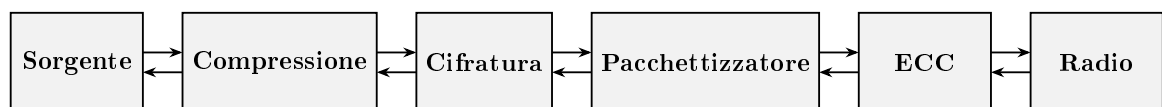


Figura 2.2: Pipeline di esempio

Questo concetto si traduce in C++ tramite un'interfaccia, dove ogni blocco può accettare un messaggio da un altro blocco.

```
1  template<typename message>
2  class Messenger{
3      public:
4          Messenger() = default;
5          virtual ~Messenger(){}
6          virtual bool input(message& m) = 0;
7  };
```

Snippet 2.1: L'interfaccia dedicata allo scambio di messaggi nella catena.

Il messaggio è una classe che mantiene un array di byte, implementando solo lo spostamento senza copia. Questa viene negata al fine di promuovere il trasferimento della proprietà dell'oggetto, evitando perdite di performance dovute alla copia profonda e possibili memory leaks. Per assicurare la corretta gestione della memoria si preferisce l'utilizzo di `std::unique_ptr`.

```

1  class Message{
2      public:
3      Message(unsigned int size) : data(new uint8_t[size]),
↪  length(size){}
4      virtual ~Message(){}
5
6      // Costruttore di spostamento
7      Message(Message &&other) noexcept : data(std::move(other
↪  .data)), length(other.length) {
8          other.length = 0;
9          other.data = nullptr;
10     }
11
12     // Operatore di assegnazione per spostamento
13     Message& operator=(Message &&other) noexcept {
14         if (this != &other) {
15             data = std::move(other.data);
16             length = other.length;
17             other.length = 0;
18             other.data = nullptr;
19         }
20         return *this;
21     }
22
23     // Disabilita il costruttore e operatore di copia
24     Message(const Message&) = delete;
25     Message& operator=(const Message&) = delete;
26
27     std::unique_ptr<uint8_t[]> data;
28     unsigned int length;
29 };

```

Snippet 2.2: Classe messaggio.

Successivamente si definiscono tipi diversi per la stessa classe messaggio, in base al ruolo assegnato.

Questa suddivisione permette di distinguere i dati prima e dopo la segmentazione, rendendo impossibile confondere dati di basso livello, parziali e potenzialmente inaffidabili, relativi ai moduli radio, con i pacchetti corretti e completi dei livelli superiori. La trasformazione da uno all'altro avviene all'interno del blocco *Pacchettizzatore* (Paragrafo 2.4).

```

1  // Suddivisione dei messaggi in base al ruolo
2  typedef Message TunMessage;
3  typedef Message RFMessage;

```

Snippet 2.3: Divisione messaggi in base al ruolo.

Infine un generico blocco della catena: definito come un oggetto che accetta uno specifico tipo di messaggio (può anche ricevere delle impostazioni, come mostrato nel capitolo 5).

```

1      template<typename message>
2      class SystemInterface : public Messenger<message>, public
    ↪ SettingsCompliant {
3
4      public:
5      SystemInterface() = default;
6      virtual ~SystemInterface(){}
7
8      virtual inline void input_finished(){}
9      virtual inline bool input(message& m) = 0;
10     virtual inline bool input(std::vector<message> &ms) {
11         for (auto& m : ms)
12             input(m);
13
14         return true;
15     }
16     virtual inline bool input(std::deque<message> &ms) {
17         for (auto& m : ms)
18             input(m);
19
20         return true;
21     }
22
23     virtual inline void apply_settings(const Settings &
    ↪ settings) override {
24         SettingsCompliant::apply_settings(settings);
25     }
26     virtual inline void stop_module() = 0;
27 };

```

Snippet 2.4: Generico elemento della catena.

2.2 Il blocco Sorgente

La sorgente dati è definita come un oggetto che eroga e riceve un flusso di sequenze finite di byte dette frame. Alcuni esempi pratici di sorgente: Rete LAN, CAN BUS, UART, SPI, I2C... La rete LAN è l'implementazione privilegiata nella fonte [1] utilizzando l'interfaccia TUN di GNU/Linux con la quale viene creata un'interfaccia di rete nel sistema con un suo indirizzo IP. I pacchetti verso e da quell'indirizzo passeranno attraverso il blocco sorgente. L'adozione di TUN rispetto a TAP è motivata da una maggiore efficienza nel trasporto dei pacchetti IP [3].

2.3 Il blocco Radio

L'interfaccia radio differisce dalla sorgente rispetto al mezzo: non più logico ma fisico. Impone una validazione della ricezione che può avvenire in due modalità: direttamente nel modulo radio alla ricezione oppure nel livello precedente al Pacchettizzatore (ECC, esempio in Figura 2.2).

2.3.1 Correzione degli errori del livello fisico

Gli errori del livello fisico sono stati investigati alla fonte [1], dove è disponibile un confronto tra le due modalità appena descritte per la correzione degli errori: implementazione del modulo radio di riferimento e correzione via software con *RS*, tramite il test dei *bit flip*. La caratterizzazione **rivela che, in condizioni ottimali, l'implementazione del produttore con il controllo del CRC [2] può scartare fino a circa il 25% dei pacchetti ricevuti** poiché contengono almeno un bit corrotto. Questa percentuale è interamente correggibile utilizzando Reed Solomon, seppur vada notato che rappresenta i pacchetti corrotti tra quelli ricevuti e non la totalità dei pacchetti persi.

Possono esistere dei pacchetti non ricevuti dal modulo radio, impossibili da correggere via software. Compatibilmente con la filosofia del progetto, la percentuale appena mostrata giustifica largamente l'implementazione dell'algoritmo RS come estensione generica del blocco Pacchettizzatore, in modo da aumentare l'efficienza complessiva del sistema anche con altri moduli radio. Va notato che questo è un passaggio cruciale rispetto a quanto detto in 1.2.2 *Astrazione: la pila OSI* per questioni di riutilizzabilità: reimplementare la soluzione appena dimostrata è costoso e lo è ancora di più reimplementare l'algoritmo HARQ mostrato nel capitolo 3.

Tabella 2.1: Test senza ECC

Kbps	Bit validi	Bit invertiti	Pacchetti corrotti	% scartati
752	92300	1748	478	16.2%
730	88691	2614	707	24.7%

Tabella 2.2: Test con RS ECC

Kbps	Bit validi	Pacchetti corrotti
1521	118880	0
1540	120380	0

Il calcolo è stato eseguito utilizzando il valore *Kbps* per ricavare i *pacchetti totali* ed il valore *pacchetti corrotti* per ricavare la percentuale dei *pacchetti scartati*, tenendo presente che un singolo pacchetto è composto da 32 byte.

2.3.2 Test dei bit flip

Il test valuta quantità e modalità di inversione dei bit in ricezione. Requisito per l'esecuzione: trasmettitore e ricevitore che permettano la ricezione grezza. Si imposta il trasmettitore per l'invio continuo di pacchetti identici formati da zero e uno alternati. Confrontando la ricezione con il pacchetto atteso, è possibile ottenere una **panoramica combinata sullo stato dell'hardware e della congestione del mezzo utilizzato**.

2.3.3 Adattatore USB

È possibile utilizzare il software anche su dispositivi che non dispongono di GPIO (come un Personal Computer) tramite un adattatore composto da un Raspberry Pi Pico abbinato a due moduli nRF24L01+. [1] Il microcontrollore implementa una parte del blocco radio e si coordina via USB con la sua controparte software, ultimo blocco della catena, assicurando la configurazione e il trasporto dei dati.

È interessante notare come errori nel trasporto siano ancora coperti dall'encoding RS.

2.4 Il blocco Pacchettizzatore

Il Pacchettizzatore è l'unico blocco della catena in grado di scambiare due tipi diversi di messaggio, dividendola quindi in due sezioni: basso e alto livello.

Il primo presuppone una comunicazione formattata per l'uscita verso il modulo radio, mentre la sua ricezione è considerata inaffidabile (da verificare e correggere).

Il secondo è riservato a scambio e lavorazione di frame interi (ricezione, cifratura, compressione) in modo bidirezionale.



Figura 2.3: Flusso dati del Pacchettizzatore

2.4.1 Funzionamento del Pacchettizzatore

Questo blocco all'effettivo è diviso in due elementi logici: una **logica di segmentazione** e una **logica ARQ o HARQ**; la struttura del software tratta questi due elementi come un solo modulo, pertanto sono inseriti nello stesso gruppo. Ad esempio, è possibile scegliere di non usare ARQ o HARQ ma una certa tecnica chiamata *character stuffing*, in questo caso non avrebbe senso parlare di segmentazione; a tal riguardo essa è critica nel rendere il frame "digeribile" dal modulo radio in uso (diversi moduli radio presentano diversi *MTU*), combinata con la logica ARQ la quale aggiunge un *header* ad ogni segmento si può ricomporre la serie disordinata ricevuta via radio e chiedere la ritrasmissione in modo selettivo prima di scartare il frame parziale.

Nel caso HARQ, la logica è la medesima, ma ogni segmento ha dei byte di *trailer* dedicati alla correzione degli errori (definito FEC [4]); si dice *HARQ con tempo limitato* quando un frame può essere scartato se impiega più di un tempo predeterminato per la trasmissione.

Quest'ultima caratteristica è vitale se il sistema finale deve lavorare in condizioni di attualità massima (Esempio: dispositivi radiocomandati).

2.5 Blocchi aggiuntivi: Cifratura e Compressione

Al momento della stesura, il software finale non include compressione inline o cifratura dei frame, anche se **la cifratura è comunque presente nel caso di traffico VPN/SSH/HTTPS**. È possibile comprimere frame per frame utilizzando algoritmi come **LZ4, Heatshrink e ROHC**, ma pongono delle ulteriori sfide legate al superamento della MTU nel caso di traffico incompressibile (spesso cifrato o già compresso), o sfide legate alla sincronizzazione nel caso di algoritmi *stateful* come ROHC che necessitano di consegna in sequenza e senza perdite.

Ovviamente, poiché la cifratura punta a massimizzare l'entropia del messaggio, il passo della compressione deve sempre essere eseguito a monte.

2.6 Ottimizzazione del software

Di seguito le ottimizzazioni adottate al fine di massimizzare le prestazioni del sistema e garantire la robustezza del codice.

Elementi del C++ moderno: L'impiego di *smart pointers* garantisce la gestione automatica della memoria, prevenendo *memory leaks*. L'integrazione delle *Move semantics* [5] permette di trasferire messaggi senza copie onerose.

Multithreading: Per parallelizzare l'elaborazione degli I/O e la logica ARQ si fa uso di: *thread pooling*, paradigma produttore-consumatore (basato su *std::deque*) e meccanismi di sincronizzazione tramite mutex e variabili di condizione.

Azioni basate su interrupt: Il flusso dati comincia sempre all'inizio o alla fine della catena, con interrupt e lavorazioni a cascata che minimizzano la latenza.

Astrazione ed ottimizzazione delle chiamate: L'architettura utilizza il passaggio per riferimento costante al fine di azzerare l'overhead di copia. L'impiego di funzioni *inline virtuali* riducono il costo computazionale dell'astrazione. La struttura deve assicurare la risoluzione di uno specifico problema per tutte le classi figlie successive, che quindi si concentrano solo sulle responsabilità più funzionali.

Esempio: la classe virtuale base Packetizer.h [1], gestisce l'inizializzazione e scambio di messaggi con gli elementi della catena inferiori e superiori del blocco Pacchettizzatore, lasciando allo sviluppatore il solo compito di implementare la propria logica tramite le funzioni virtuali pure:

```

1 virtual inline void process_tun(TunMessage &m) = 0;
2 virtual inline void process_packet(RFMessage &m) = 0;
3 virtual inline void apply_settings(const Settings &settings)
   ↳ override { /** Parzialmente implementata */};
4 virtual inline unsigned int get_mtu() = 0;
```

Snippet 2.5: Funzioni da implementare in un pacchettizzatore.

Capitolo 3

Segmentazione e HARQ

3.1 La segmentazione

3.1.1 Messaggi dalle dimensioni sconosciute

Si rende necessario valutare la compatibilità dei messaggi con il blocco radio. In genere le applicazioni più semplici non hanno necessità di suddividere il *frame*, ma in queste casistiche l'intero software della trattazione risulta superfluo. D'altro canto, applicazioni di alto livello possono affacciarsi su reti più complesse e sicuramente presenteranno frame più lunghi di quanto un modulo radio possa accettare. Nella pratica, i casi dove il blocco radio possa accettare un frame per intero sono limitati.

Per riferimento, un nRF24L01 può accettare fino a 32 byte per pacchetto con un massimo di tre pacchetti per trasmissione.

3.1.2 Modalità di segmentazione

Per segmentare il frame è sufficiente definire una dimensione comoda.

Poiché il problema posto fissa solamente la lunghezza massima del frame che non è necessariamente un multiplo della dimensione del segmento (è il caso per ARQ e HARQ al paragrafo 3.2), allora il segmento in testa o in coda risulterà avere dimensione uguale o minore.

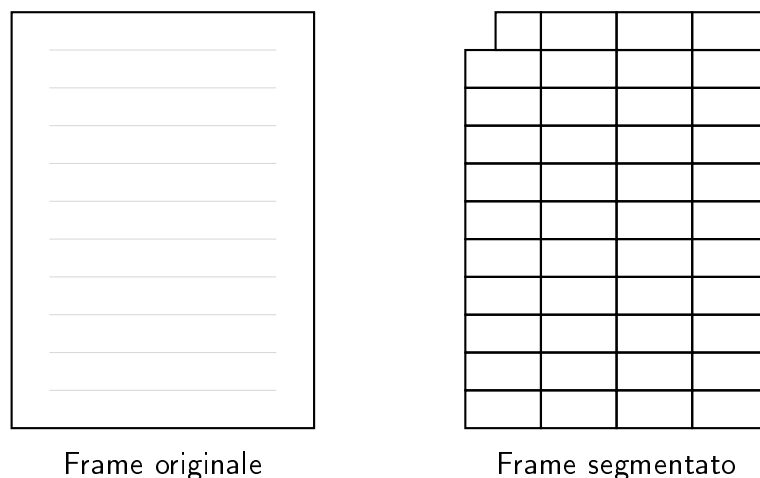


Figura 3.1: Segmentazione del frame.

3.2 Algoritmo ARQ

Suddiviso il frame, si può procedere in due modi: invio con o senza ricevuta di ritorno.

Caso senza ricevuta di ritorno

Sequenze di byte sono trasmesse con allegato CRC o FEC e opzionalmente le informazioni per riposizionare il segmento nel buffer di ricezione. È un modo di invio debole dove la probabilità di errore cresce insieme al numero di segmenti da inviare: perdere un solo pacchetto può invalidare l'intero messaggio.

Caso con ricevuta di ritorno

Si trasmettono sequenze di byte con allegato CRC, informazioni sul posizionamento nel buffer di ricezione e sul numero di segmenti del messaggio completo; implementando un sistema di conferma o recupero dei segmenti persi tramite un meccanismo con **ACK/NACK**: *Acknowledgement* o *Negative Acknowledgement*. Uno degli algoritmi che utilizzano questi elementi logici è **ARQ**: *Automatic Repeat Request*.

Nel caso in cui i pacchetti ARQ includessero anche dei byte per FEC, allora l'algoritmo è detto **HARQ**: *Hybrid Automatic Repeat Request*. Grazie a questa combinazione, spesso è necessario recuperare solo i pacchetti persi completamente invece di chiedere una seconda volta anche quelli parzialmente corrotti. In caso di utilizzo del solo CRC è necessario richiedere anche questi ultimi.

3.2.1 Modalità di funzionamento

Lo sviluppo della logica ARQ implica il caso con ricevuta di ritorno, supponendo per semplicità il trasferimento di un solo messaggio (frame) per volta.

I passaggi sono i seguenti.

Preparazione del messaggio in ingresso

1. Ricezione del messaggio dal livello superiore.
2. Suddivisione in segmenti, quindi numerazione.
3. Aggiunta di header che identifichino il messaggio in uscita e la posizione del segmento, in modo da non far perdere di vista la finestra di trasmissione al ricevitore.
4. Aggiunta di trailer che permettano la verifica e/o correzione del contenuto di ogni pacchetto radio.

Flusso di lavoro del trasmettitore

1. Appena la trasmissione è pronta, si inviano tutti i pacchetti in sequenza, ricordando che l'ultimo pacchetto contiene informazioni aggiuntive sul messaggio, permettendone la ricostruzione e conferma o richiesta dei segmenti mancanti.
2. A trasmissione completata, sono avviati due timer: il primo indica la scadenza del frame (solo ARQ con scadenza temporale) e il secondo, più breve, causa la ritrasmissione dell'ultimo segmento nell'ipotesi che questo o la risposta siano andati persi.
3. Se si riceve ACK e i dati che contiene sono coerenti con il messaggio in uscita, lo si conferma e la trasmissione avanza al prossimo. Se l'ACK non contiene dati coerenti si può scegliere di ignorarla o ritrasmettere l'intero messaggio.
4. Se si riceve NACK, all'interno dello spazio dati è disposto un elenco di segmenti da ritrasmettere. Se si trasmette un frame per volta, bisogna verificare la validità delle richieste prima di procedere alla ritrasmissione individuale (dovranno essere tutte riferite allo stesso frame).

Flusso di lavoro del ricevitore

1. Il ricevitore dispone di un buffer di dimensione massima (MTU), uno spazio per marcare i segmenti ricevuti, la dimensione del messaggio e altri campi identificativi della finestra del trasmettitore (vedi sezione 3.2.2)
2. Alla ricezione di un pacchetto contenente solo dati, copia il contenuto nello spazio corrispondente nel buffer.
3. Alla ricezione di un pacchetto marcato come "ultimo", verifica la plausibilità dei contenuti:

Dimensione del messaggio entro MTU

Il segmento finale contiene un numero di byte coerente con la sua capacità

Una volta convalidato, il buffer di ricezione va aggiornato con il contenuto del segmento.

Il pacchetto finale deve contenere almeno queste informazioni: lunghezza totale del messaggio, byte di dati contenuti nel segmento finale, segmenti totali trasmessi ed il CRC del frame per la verifica dell'integrità al ricevitore. Se presenti, si copiano gli ultimi byte di dati nel buffer.

Verdetto ACK o NACK

Le possibili risposte a pacchetti "ultimi" sono definite nella seguente tabella.

Segmenti mancanti	CRC	Esito
zero	Verificato	ACK: il buffer va consumato una volta
zero	Non verificato	NACK: tutti i segmenti del messaggio
almeno uno	-	NACK: solo mancanti

Tabella 3.1: Gestione risposte ARQ.

La risposta ACK contiene dati che indicano la corretta ricezione al trasmettitore, ovvero: identificativo del messaggio, CRC e numero di segmenti.

Da tenere presente che è possibile ricevere di nuovo lo stesso segmento finale a cui va risposto ACK senza consumare di nuovo il buffer.

Ogni ricezione con un identificativo diverso da quello attuale deve causare un reset o cambio di buffer. È estremamente importante evitare di mischiare segmenti appartenenti a gruppi diversi.

3.2.2 Formato dei pacchetti

La struttura di un pacchetto HARQ è composta da tre sezioni: header, dati e trailer.

Header : Responsabile dell'instradamento e indicizzazione dei segmenti, distingue traffico dati dai messaggi di sistema (ACK e NACK). (1-2 byte)

Dati : Il segmento di dati da trasportare.

Trailer : Garantisce la discriminazione dati-rumore. Include un byte CRC e dei byte di ridondanza per la codifica RS, capace di correggere fino a $t/2$ byte con errori [6]. La protezione viene applicata calcolando il CRC su header e dati, seguito dal calcolo di RS sull'intero pacchetto.



Figura 3.2: Struttura di un pacchetto HARQ

3.2.3 Interno dell'Header

L'header deve essere una struttura allineata al byte (unità dei messaggi) a cui dedicare idealmente un solo byte. Nella pratica è necessario utilizzare due byte, dove il secondo estende la sezione ID.

Il bitset relativo al primo byte è così composto:

ID (2 bit) : Conosciuto come *Sequence Number*, identifica un frame in uscita.

La combinazione con ID zero è riservata ai pacchetti di sistema che non sono mai nulli, poiché durante la caratterizzazione degli errori eseguita sui moduli nRF24L01+ è risultata un'alta frequenza di ricezioni composte da soli zeri e questi non solo risultano validi per entrambi CRC ed RS ma potrebbero essere comuni durante la trasmissione dati, per cui si è scelto di eliminare il problema in modo strutturale. Questo rende possibile trasmettere anche altri messaggi di protocollo insieme ai dati (esempio: configurazione remota).

Un Sequence Number così ridotto può non riconoscere efficacemente segmenti di messaggi diversi (caso con timeout di esattamente $ID_{max} - 1$ messaggi), per cui vi è stato abbinato un secondo byte contenente il CRC del messaggio intero invece di un semplice *SN*, questo svolge due funzioni contemporaneamente: raggruppa i segmenti dello stesso frame e ne permette la verifica a ricezione completa.

Indice Segmento (5 bit) : Definisce la posizione dei dati contenuti nel pacchetto. Con 5bit è possibile raggiungere un $MTU_{max} = (2^{bit\ segmento} * Capacità) - 1 = 927\ byte$, con $Capacità = byte\ Pacchetto - byte\ Header - byte\ ECC = 29$. Dato che i segmenti sono tutti della stessa lunghezza eccetto il primo o l'ultimo (si suppone l'ultimo per semplicità) la posizione nel buffer in trasmissione o ricezione si definisce $Posizione = Indice \times Capacità$. In questo modo l'ultimo segmento indicherà contemporaneamente la posizione del segmento dati asimmetrico e la dimensione del frame originale.

Flag "ultimo pacchetto" (1 bit, flag) : Indica se il pacchetto ricevuto è un segmento dati o è l'ultimo della trasmissione. Se ultimo, questo conterrà informazioni aggiuntive come descritto in 3.2.1 *Flusso di lavoro del ricevitore*.

ID	Indice segmento	Flag
----	-----------------	------

Figura 3.3: Suddivisione del primo byte di Header

Considerazioni sul secondo byte di header: Sebbene l'integrazione del CRC permetta teoricamente la rimozione del campo ID, la sua conservazione è preferibile per due motivi:

1. Aumentare il numero di segmenti permette un maggior trasferimento dati con minor overhead, ma rallenta la trasmissione per via dei maggiori errori accumulati che eccederebbero la capacità di segnalazione di un singolo NACK.
2. La probabilità di collisione tra frame successivi aumenterebbe poiché è possibile avere due frame a breve distanza con lo stesso CRC8 (solo 256 valori possibili), causando un'indecisione e bloccando la comunicazione. Mantenendo il campo ID la probabilità di collisione è ridotta: è necessario avere due frame con lo stesso CRC, posti esattamente a distanza $D = 2^{bit\ ID} - 1$ e il ricevitore deve perdere esattamente D frames per non accorgersene.

3.3 Flusso di lavoro con disturbi

La struttura descritta in questo capitolo è resistente relativamente a duplicati, perdite e corruzione seppure in modo parziale senza algoritmi come RS.

La figura seguente fornisce una panoramica del comportamento dell'algoritmo sul campo, simulando alcuni disturbi.

Da notare che il duplicato sovrascrive la memoria, ma non necessariamente causa un reset del buffer di ricezione.

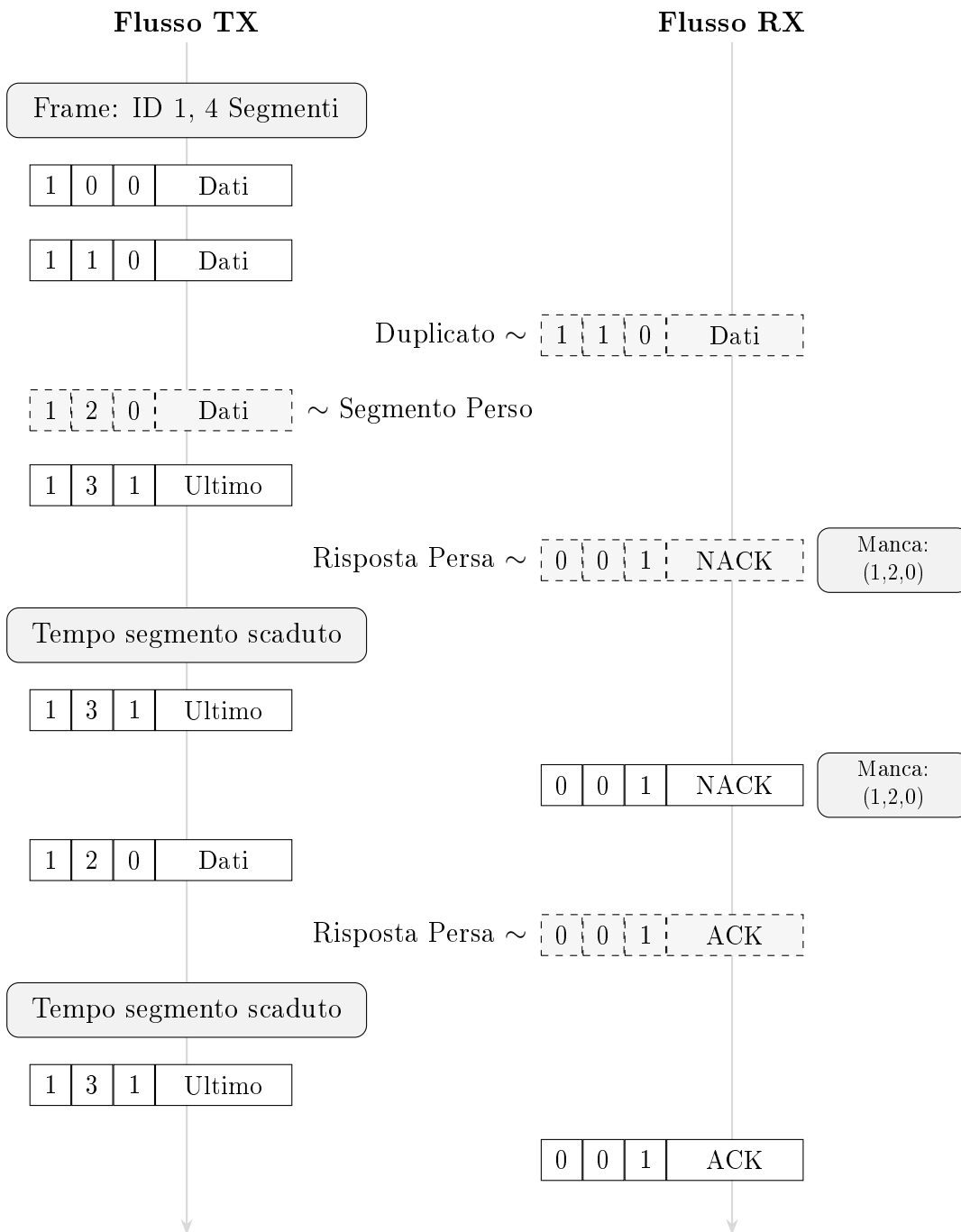


Figura 3.4: Flusso di comunicazione ARQ con disturbi

Capitolo 4

Gestione multithreaded dei dati

Definita la struttura, è necessario affrontare le scelte architetturelle relative allo scambio concorrente di pacchetti, le quali incidono direttamente sulle prestazioni del sistema. L'obiettivo finale di queste scelte è l'elaborazione di dati con la massima efficienza e reattività, riducendo l'overhead computazionale.

4.1 Considerazioni sulle strutture disponibili

Un'architettura *event-driven* risulta significativamente più efficiente rispetto a una basata sul *busy waiting*; la seconda risulta più adatta per software di simulazione rispetto ad un driver radio; mentre la prima permette di riattivare la CPU solo quando è necessario intervenire.

La soluzione finale può comprendere l'utilizzo di tecniche come:

Interrupt: Segnale asincrono generato da un evento hardware, richiede un intervento immediato dal kernel. L'esecuzione deve essere il più breve possibile: va utilizzato strettamente per leggere ed accodare dati.

Poll: Singolo ciclo con lettura di stati predefiniti. Ogni ciclo contiene uno o più eventi da processare, se assenti, la CPU dorme.

io_uring: Interfaccia asincrona che fa uso di code circolari (*Submission* e *Completion Queues*). Minimizza il numero di chiamate di sistema rispetto a **Poll**, riducendo la latenza e l'overhead di *context switching*.

L'integrazione con operazioni di *Scatter/Gather* tramite la struttura **iovec** permette di gestire buffer di memoria non contigui come una sola unità logica. Si parla di *Zero-Copy Structure*: i dati in ingresso sono inseriti direttamente negli spazi preallocati dei segmenti in uscita. Non si effettuano altre copie: il sistema si occupa di inserire header e trailer negli spazi riservati.

Thread	Sistema	Problemi
Sincrono	Poll	Perdita di reattività
Doppio	Misto: Interrupt e Poll	Race conditions

Tabella 4.1: Confronto tra approccio sincrono e concorrente per il sistema.

4.2 Soluzione implementata

L'approccio intrapreso nella seconda versione del software fa uso di quattro thread con *deque*. Responsabilità:

1. Leggere frame dalla sorgente TUN ed inserirli in coda di lettura per il pacchettizzatore
2. Leggere frame dalla coda in ordine di arrivo, processarli uno per volta e seguire il ciclo di vita fino a scadenza. Include il rinvio di un segmento allo scadere del timer. Poiché non si conosce a priori il numero di messaggi da scambiare, i pacchetti in uscita sono gestiti tramite liste collegate.
3. Leggere dal modulo radio alla ricezione di interrupt, poi inserire ogni pacchetto in coda di lettura per il pacchettizzatore.
4. Leggere i pacchetti provenienti dal modulo radio e processarli in ordine.

La modalità di operazione descritta assicura un buon utilizzo riferito a una CPU quad core (target: Raspberry Pi 3, 4, 5, Zero 2).

Ogni race condition è gestita tramite mutex e variabili condizione: un metodo di attesa con risveglio su notifica.

Le liste doppiamente collegate non trasportano dati, ma riferimenti ai dati (tramite *std::move*). Quindi non si riscontrano perdite di prestazioni dovute alla copia, ma potrebbero verificarsene altre per natura della lista collegata dove gli elementi da leggere non sono sequenziali in memoria, causando cache misses.

4.3 Soluzione della terza versione

L'approccio utilizzato nella terza versione del software, attualmente in fase prototipale, è un misto interrupt-poll con *Scatter/Gather*: l'interrupt notifica il momento in cui i dati sono pronti per il recupero dal modulo radio evitando il controllo continuo, mentre poll permette lo scambio di vari eventi come i seguenti:

```

1  enum Eventi_Poll{
2      FRAME_PRONTO ,
3      RADIO_PRONTA ,
4      FRAME_COMPLETATO ,
5      FRAME_SCADUTO ,
6      SEGMENTO_SCADUTO
7  };

```

Snippet 4.1: Enumerazione degli eventi poll.

L'approccio con Scatter/Gather permette la scrittura del frame direttamente negli spazi preassegnati in memoria (Zero-Copy Structure) che prevedono header e trailer nei byte immediatamente precedenti e successivi per ogni segmento. La lettura dei segmenti in uscita è sequenziale ed avviene in un blocco dati allineato alla cache, garantendo le massime prestazioni di esecuzione.

4.4 Gestione dell'ambiente multithreaded

Segue una breve introduzione riguardante gli elementi dedicati a sincronizzazione e gestione degli accessi in contesti multithreaded.

4.4.1 Variabile Atomic

Una variabile di tipo `std::atomic` assicura modifica e lettura di un solo dato protette da race conditions. Le operazioni risultano più rapide rispetto al blocco del mutex, ma sono adatte solo per piccole operazioni definite atomiche.

Maggiori informazioni alla fonte [7].

4.4.2 Mutex e Unique Lock

Il componente `mutex` sta per *mutually exclusive*. Si utilizza per bloccare una risorsa, solitamente inserito all'inizio di una funzione insieme a `std::unique_lock`. Questo resta bloccato fino a fine *scope*. Più informazioni alle fonti [8] e [9].

```
1 std::mutex nostro_mutex;
2
3 void procedura_con_caller_multipli(){
4
5     {    // Creazione di uno scope aggiuntivo
6
7         // Riga bloccante fino a quando
8         // la nostra istanza non blocca il mutex
9         std::unique_lock<std::mutex> lock(nostro_mutex);
10
11         // Le operazioni che richiedono il mutex vanno qui
12
13     }    // Fine scope: sblocco del mutex
14     // NOTA: in assenza di questo scope {}
15     //         il lock viene liberato a fine funzione.
16
17     // Altre operazioni che non richiedono il mutex
18     // . . .
19 }
```

Snippet 4.2: Esempio di mutex con unique lock.

4.4.3 Le variabili condizione

La `std::condition_variable` abilita l'attesa di un certo evento in modo efficiente (*graceful*).

Maggiori informazioni alla fonte [10].

Esempio pratico: il thread descritto nel punto 2 della sezione 4.2 attende la ricezione di elementi nella coda. L'attesa comincia nella chiamata `next_frame()`, bloccante fino a quando non ci sono frame disponibili. La `std::condition_variable` si sincronizza sulla condizione `!outgoing_frames.empty()`, la quale sarà controllata al risveglio del thread che avviene in seguito alla chiamata `frames_cv.notify_one()`

```
1 std::mutex outgoing_frames_mtx;
2 std::condition_variable frames_cv;
3
4 std::deque<Frame> outgoing_frames;
5
6 // Aggiunge un frame alla coda
7 // Chiamante: thread dedicato alla sorgente
8 inline void add_frame_to_queue(Frame& f){
9     std::unique_lock<std::mutex> lock(outgoing_frames_mtx);
10    outgoing_frames.push_back(std::move(f));
11    frames_cv.notify_one();
12 }
13
14 // Attesa di un frame disponibile
15 // Funzione semplificata: l'originale gestisce anche il caso
16 // di chiusura del programma
17 inline Frame next_frame() {
18     std::unique_lock<std::mutex> lock(outgoing_frames_mtx);
19     frames_cv.wait(lock, [&] {
20         return !outgoing_frames.empty();
21     });
22
23     auto f = std::move(outgoing_frames.front());
24     outgoing_frames.pop_front();
25     return f;
26 }
```

Snippet 4.3: Esempio di condition variable.

Capitolo 5

Sistema di configurazione utente

Risulta molto utile fornire un sistema di configurazione semplificato che non richieda la modifica dei sorgenti del programma né la ricompilazione. È preferibile cambiare configurazione e moduli in modo dinamico, al più con un riavvio del software.

5.0.1 Configurazioni a Compile time e Runtime

Il sistema appena descritto implica una configurazione a runtime, quindi allocazione dinamica. Questa comporta una maggiore difficoltà di realizzazione del software e la conseguente allocazione potenzialmente non ottimale può portare a *cache misses* con perdita di performance. Le configurazioni a compile time permettono invece un'allocazione statica e sono ottimizzabili dal compilatore.

Nella realizzazione della versione 2 del software si è seguito un approccio piuttosto estensivo: è stata permessa la configurazione della quasi totalità dei parametri non costanti.

5.0.2 Formato del file di preset

Il software può accettare più file di configurazione (preset) contemporaneamente, in modo tale da dedicare ogni file a uno specifico modulo.

Esempio di avvio del programma:

```
./rf24tunlink
    presets/CONFIGURAZIONE_SORGENTE_A
    presets/CONFIGURAZIONE_HARQ
    presets/CONFIGURAZIONE_RADIO_A
    presets/CONFIGURAZIONE_REEDSOLOMON
    presets/RUOLO_PRIMARIO o presets/RUOLO_SECONARIO
```

Dove ogni file di preset presenta il formato:

```
# Descrizione impostazione
# Altra riga di descrizione se necessaria
nome_impostazione = valore

# Impostazione booleana
altra_impostazione = si/no
```

5.1 Struttura della classe Impostazioni

Si inseriscono i valori in una classe (o *fat struct*) che preveda una utility per la lettura dei file di preset e la sua esportazione con il formato descritto. Al fine di organizzare i valori per l'accesso (entries), vi sono diverse modalità di implementazione, una delle più semplici è la seguente:

```

1 enum tipi_di_dato{bool, string, uint8, uint32, integer };
2 class Settings{
3     // Compilare questi vettori
4     void* riferimenti[] = {
5         &nome_impostazione,      . . . .
6     };
7     std::string nomi_impostazioni[] = {
8         "nome_impostazione",      . . . .
9     };
10    tipi_di_dato tipi_impostazioni[] = {
11        integer,      . . . .
12    };
13    std::string descrizione_impostazioni[] = {
14        "Indica il valore dell'impostazione",      . . . .
15    };
16    public:
17        // Impostazione in memoria di esempio
18        int nome_impostazione = valore di default;
19 };
20
21 void utility_leggi_preset_da_file(const std::string filename,
22     ↪ Settings &dove_applicare);
23 void scrivi_su_file(std::ostream output, const Settings
24     ↪ impostazioni_attuali);

```

Snippet 5.1: Esempio classe impostazioni.

L'approccio mostrato è funzionalmente equivalente al seguente, differendo nella disposizione in memoria:

```

1 struct entry {
2     void* variabile;
3     std::string nome;
4     tipi_di_dato tipo;
5     std::string descrizione;
6 };
7 // Compilare le struct
8 entry impostazioni[] = {
9     {&nome_impostazione, "nome_impostazione", integer, "Indica
10     ↪ il valore dell'impostazione"},      . . . .
11 };

```

Snippet 5.2: Esempio classe impostazioni equivalente.

5.2 Interfaccia di configurazione

Al fine di assicurare l'accesso alle impostazioni più recenti, si fa uso di un'interfaccia implementante la chiamata *apply_settings*. Questa modalità permette deallocazione e reimpostazione di ogni elemento evitando il riavvio del processo (*graceful reload*).

```

1 class SettingsCompliant {
2     protected:
3     const Settings* settings = nullptr;
4     public:
5     SettingsCompliant() = default;
6     virtual ~SettingsCompliant() {
7         settings = nullptr;
8     }
9     virtual inline void apply_settings(const Settings &settings)
10    {
11        this->settings = &settings;
12    }
13    inline const Settings* current_settings() const {
14        return (settings);
15    }
16 };

```

Snippet 5.3: Interfaccia oggetto per le impostazioni.

La classe che implementa l'interfaccia deve gestire la relativa chiamata nel modo seguente:

```

1 class Oggetto : public SettingsCompliant {
2     virtual inline void apply_settings(const Settings &settings)
3     ↪ override {
4         // Chiamata alla classe superiore
5         SettingsCompliant::apply_settings(settings);
6
7         /** Rilascio e chiusura se necessario */
8
9         /** Ricalcolo e allocazione */
10    }
11
12 };

```

Snippet 5.4: Modalità di applicazione delle impostazioni per un oggetto.

Lo stesso vale per le classi derivate: ognuna deve disporre una chiamata alla propria superclasse, garantendo l'applicazione completa e pulita delle nuove impostazioni. L'ultima chiamata dello stack ne assicura il salvataggio all'interno di ogni modulo.

Capitolo 6

Il Circuito Stampato

È stato implementato un *test bench* per poter effettuare prove sul campo.

6.1 Requisiti del Banco di Prova

Devono sussistere le condizioni affinché:

- Il modulo radio sia fissato stabilmente.
- Si assicuri la stabilità dell'alimentazione e la copertura dei requisiti massimi di potenza erogata.
- Si raggiunga la velocità di trasmissione massima del modulo radio attraverso il mezzo fisico di collegamento tra Raspberry Pi e il modulo radio.

Necessario per la velocità di trasmissione: isolamento della linea dai disturbi e riduzione dei riflessi nel collegamento.

- I collegamenti siano il più brevi possibile.

Per garantire la consistenza delle prove è preferibile realizzare un PCB dedicato rispetto ad un circuito su breadboard o scheda millefori.

6.2 Realizzazione del PCB

Accorgimenti necessari per soddisfare i requisiti punto per punto:

1. È sufficiente installare un socket da 2.54 mm .
2. Dimensionare opportunamente l'alimentazione e disporre delle tracce larghe per collegare alimentatore e radio (circa 1.6 mm).

Per garantire la stabilità si prevedono due condensatori posizionati il più vicino possibile ai collegamenti per l'alimentazione di ogni modulo: un condensatore ceramico di piccole dimensioni (MLCC) e un elettrolitico (alternativamente un condensatore al tantalio) più capiente. Il primo è un filtro per il rumore elettrico ad alta frequenza, il secondo stabilizza la tensione nelle vicinanze del modulo.

- 3 - 4. I collegamenti devono essere brevi e della stessa lunghezza (*length matching*), per assicurare che segnali su più bus transitino allo stesso tempo. Ogni percorso non deve avere angoli retti. Opzionalmente, non richiesto per SPI, considerare tecniche più avanzate come *impedance matching*.

Si adottano i seguenti accorgimenti per isolare le linee dal rumore:

- Piazzamento delle tracce di segnale negli strati esterni del PCB.
- *Ground pour* negli strati dei segnali e lo strato successivo o precedente. Le tracce devono sempre avere un comune continuo sotto ed ai lati.
- Le tracce non devono essere vicine nè parallele tra loro.
- Se necessario incrociare le tracce solo perpendicolarmente.
- Utilizzo di *stitching vias*: da inserire in modo uniforme tra le tracce e nel contorno dell'unità. Formano un filtro passa-alto. La frequenza di taglio dipende dalla distanza tra i *via*: lunghezze d'onda più piccole di questa distanza potranno passarvi attraverso.

La figura 6.2 mostra gli accorgimenti appena descritti.

6.2.1 Circuiti Stampati Sviluppati

Sono stati sviluppati tre tipi di circuiti stampati, ognuno con una destinazione d'uso differente. Semplice collegamento radio: coppia identica provvista di due connettori per moduli radio, un LED, un tasto ed un connettore per controller.

Collegamento per comunicazione, controllo e telemetria: coppia di schede, una scheda è la precedente e l'altra contiene gli elementi essenziali per una macchina radiocomandata.

Il terzo tipo di scheda non è disponibile pubblicamente poiché utilizzato in Scuderia UNICAS. È una configurazione ibrida contenente gli elementi necessari per collegare i sensori a bordo della vettura al ponte radio.

6.2.2 Utilizzo in Scuderia UNICAS

In Scuderia UNICAS, il ponte radio vede il suo utilizzo nella trasmissione della telemetria del veicolo al box, permettendo allo stesso tempo al pilota di comunicare in tempo reale tramite il tasto predisposto (*Push to Talk*) e additionally è anche possibile trasmettere la visuale di bordo in tempo reale, sul medesimo canale.



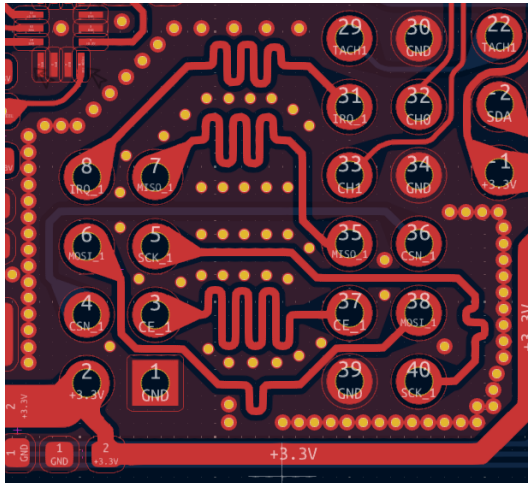
Squadra UNICAS



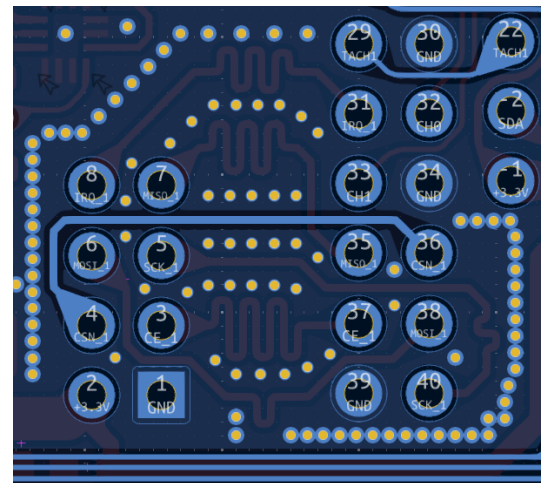
Volante e dashboard

Le figure di seguito mostrano un bus SPI che collega Raspberry Pi con il socket per nRF24L01 rispettando tutti i requisiti elencati prima, successivamente un render del PCB con l'equipaggiamento di bordo per la macchina radiocomandata.

Differenza di performance: la velocità massima del bus prima dell'applicazione degli accorgimenti descritti era di 8 MHz, mentre successivamente è stato possibile raggiungere i 16 MHz.



(a) Fronte



(b) Retro

Figura 6.2: Strati PCB per nRF24L01

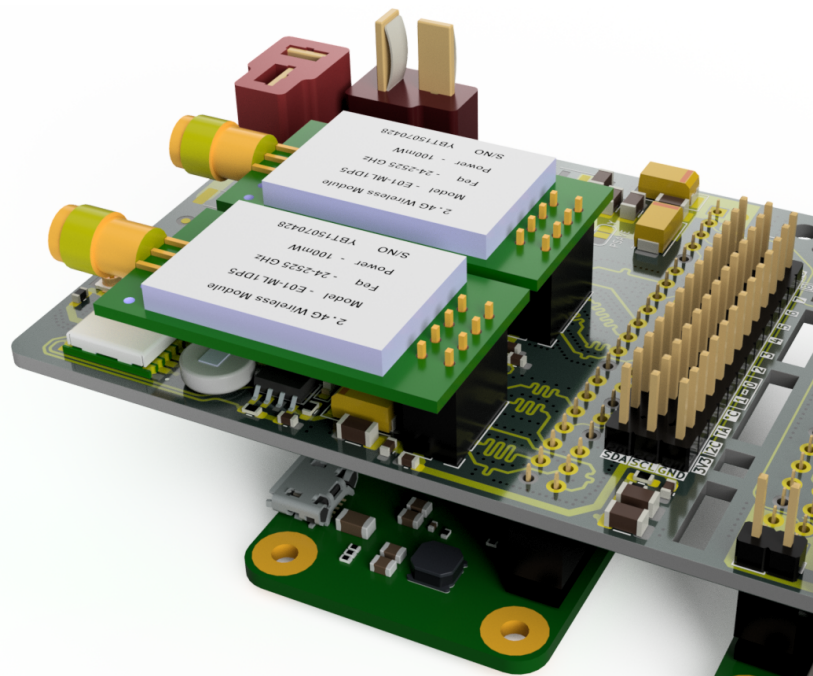


Figura 6.3: Render PCB RX

6.3 Prove sul campo

Durante lo sviluppo in Scuderia UNICAS è stata eseguita una verifica preliminare al fine di valutare la qualità della ricezione per i moduli scelti. Successivamente è stato ipotizzato un eventuale caso reale con oggetto l'autodromo di Varano De' Melegari nel quale la competizione Formula SAE è tenuta annualmente. Altre verifiche qualitative sono state effettuate in sede del team.

6.3.1 Svolgimento della prova e risultati

Sono stati impiegati due moduli radio in configurazione Half-Duplex muniti di antenna da 6 dBi, valutando il degrado del canale all'aumentare della distanza in *Line of Sight (LOS)*. Il test si è tenuto su un tratto rettilineo con avvallamenti e traffico moderato. La velocità massima attesa per il software in stadio preliminare di sviluppo (goodput di 300 kbps) è stata mantenuta per tutti i punti in *LOS* valutati fino a 900 metri dove poi la strada è terminata.

6.3.2 Ricezione negli uffici

Grazie alla configurazione Full-Duplex è stato possibile trasmettere video o audio all'interno degli uffici del team, in condizioni di *Non-Line-of-Sight (NLOS)*: la comunicazione ha attraversato le mura degli uffici per arrivare dapprima nell'officina situata alla fine di un largo corridoio adiacente, infine sino all'esterno dell'edificio.

6.3.3 Ipotesi di copertura in autodromo

In vista della competizione *FSAE 2025* sono state valutate le zone in cui la comunicazione era probabile considerando un endpoint fisso in uno dei box e l'altro sulla vettura in movimento. Le ipotesi mostrate in Figura 6.4 non sono ancora state verificate sul campo. Riconfermata la necessità di dimensionare il traffico radio.



Legenda ostruzioni:

Nero Sempre ostruito.

Bianco Sempre visibile.

Blu Ostruito solo per il lato destro dei box.

Rosso Ostruito solo per il lato sinistro dei box.

Figura 6.4: Ipotesi di copertura in autodromo.

Capitolo 7

Installazione del Software

Le seguenti istruzioni sono disponibili all'interno del file LEGGIMI alla fonte: [1]

Andrea Roccaccino. rf24tunlink: Point to Point TUN link via nRF24L01 radio modules

7.1 Piattaforma per l'implementazione

La prima implementazione del software si basa sull'ambiente GNU/Linux, considerato lo standard *de facto* per i **SBC**. Tale scelta è motivata dalla natura **open source** del sistema, che si allinea con la filosofia del progetto e permette l'accesso a una vasta documentazione ed al supporto della comunità.

7.2 Preparazione di Raspberry Pi e CMake

Seguono i passaggi da seguire, in ordine, per configurare l'ambiente di compilazione, scaricare e compilare il programma finito.

1. Modificare il *config.txt* presente in `/boot/firmware/config.txt` attivando SPI e inserendo in fondo al file: `dtoverlay=spi1-3cs` e riavviare il sistema.
Questo passaggio attiva la seconda linea SPI del Raspberry Pi in modo da collegare i due moduli radio a due bus SPI separati ed evitare qualsiasi problema di sincronizzazione dovuto agli interrupts (*race conditions*).
2. Installare gli strumenti per scaricare il software da git, per la compilazione e CMake.
In un terminale, inserire:

```
sudo apt-get install build-essential cmake make git g++ gpio -y
```
3. Poiché si è scelto di utilizzare dei moduli nRF24L01, installare la libreria **RF24 Core** dalla fonte TMRh20 e 2bndy5. *RF24: OSI Layer 2 driver for nRF24L01 on Arduino & Raspberry Pi/Linux Devices (English)*. URL: <https://github.com/nRF24/RF24> (visitato il giorno 18/01/2026)
4. Acquisire i sorgenti del software tramite clonazione del repository:

```
git clone https://github.com/LollosiSi/rf24tunlink.git
```
5. Compilare il programma tramite lo script `./build.sh` all'interno della cartella del progetto.

7.3 Il file CMakeLists.txt

Poiché la compilazione di programmi C++ è strettamente legata all'architettura destinataria, il sistema CMake si configura come uno strumento essenziale per la gestione multiplatforma del sorgente. Il file CMakeLists.txt contiene informazioni riguardanti la ricerca delle librerie nel sistema, il progetto e la sua compilazione.

Descrittore del progetto

```
project (rf24tunlink2
        VERSION 0.1
        DESCRIPTION "TUN Link Driver")
```

Flag di compilazione

```
add_compile_options(-Ofast -Wall -march=native -fno-signed-zeros
    ↪ -fno-trapping-math -frename-registers -funroll-loops)
```

Flag per debug

```
set(CMAKE_LINKER_FLAGS
    "${CMAKE_LINKER_FLAGS} -fsanitize=address")
```

Ricerca delle librerie

```
find_library(RF24 rf24 REQUIRED)
```

Raccolta file sorgenti

```
set(sourcefiles
    utils.h
    crc.h
    ${reedsolomon}
    ${rs_codec}
    ${settings}
    ${generics}
    ${dialogators}
    ${radios}
    ${packetizers}
    ${activity_signals}
)
```

Creazione eseguibile e linking

```
add_executable(${PROJECT_NAME} main.cpp ${sourcefiles})

target_link_libraries(${PROJECT_NAME} PUBLIC ${RF24} pthread)
```


Capitolo 8

Conclusioni

Si introducono alcune metriche al fine di valutare la qualità dell'esecuzione e la corrispondenza della destinazione d'uso reale con quella presentata.

8.1 Valutazione del software

La valutazione si basa su una serie di osservazioni qualitative e quantitative.

8.1.1 Valutazione qualitativa

I requisiti imposti all'interno della trattazione sono ragionevolmente soddisfatti:

- Scambio di messaggi generico e indipendente dal supporto fisico.
- Trasparenza e facilità di utilizzo.
- Integrità della ricezione garantita senza duplicati.
- Tempestività garantita con le seguenti eccezioni:
 - Condizioni di scarsa ricezione.
 - Scarto del frame se previsto in caso di accumulo in uscita o eccessivo ritardo.
- Resistenza ai disturbi.
- Collegamento di tipo Full-Duplex.
- Architettura modulare configurabile senza ricompilazione.
- Oneri di replicazione contenuti.

8.1.2 Valutazione quantitativa

La valutazione in cifre ha senso attraverso due elementi: *ping* e *goodput*.

Metrica	Unità	Min	Avg	Max
Ping	ms	1,6	2	<i>Timeout</i>
Goodput	Mbps	0,3	1,2	1,35

Tabella 8.1: Analisi delle prestazioni di rete.

Va notato che il tempo di ping subisce aumenti con disturbi e il conseguente accumulo di pacchetti. In condizioni ordinarie la coda di uscita è svuotata più velocemente di quanto sia popolata, per cui il sistema rimane stabile.

Il goodput indicato in tabella è inteso per canale ed il collegamento è simmetrico. La larghezza di banda massima per nRF24L01 è 2 Mbps.

8.1.3 Valutazione overhead

L'overhead del sistema è una misura di efficienza dell'algoritmo ARQ e HARQ descritto: definito come il rapporto tra i byte di metadati (protocollo, ridondanza) e la lunghezza del pacchetto. Il valore varia in base alla configurazione utilizzata.

Gli elementi configurabili sono i seguenti:

Header: base (1 byte) o esteso (2 byte). Il byte aggiunto incrementa la resistenza alle perdite. Sezione 3.2.3.

Trailer: numero di byte dedicati all'algoritmo di correzione degli errori. La metà di questo valore rappresenta il numero di byte recuperabili.

CRC in trailer: è possibile inserire il CRC del segmento nel trailer.

Hd.	Tr.	Recup.	Overhead	Algoritmo	Capacità di correzione
1	0	0	3,13%	ARQ	ACK invalido / livello sup.
2	0	0	6,25%	ARQ	Integrità frame
1	1	0	6,25%	ARQ con CRC	Integrità pacchetto
2	1	0	9,38%	ARQ con CRC	Integrità pacchetto e frame
1	2	1	9,38%	HARQ	ARQ + FEC
2	2	1	12,50%	HARQ	ARQ + FEC
1	3	1	12,50%	HARQ	ARQ con CRC + FEC
2	3	1	15,63%	HARQ	ARQ con CRC + FEC

Tabella 8.2: Analisi semplificata dell'overhead.

La tabella 8.2 considera la struttura del pacchetto dati in uscita, di dimensione 32 byte, con una semplificazione: il pacchetto "ultimo" presente in ogni trasmissione con frequenza variabile comporta un byte addizionale di overhead, aumentando leggermente il valore reale rispetto all'analisi fatta.

8.2 Interpretazione dei risultati

La capacità raggiunta nel caso mostrato con il modulo radio nRF24L01 è sufficiente a garantire la coesistenza di più servizi per l'utente finale, particolarmente indicati sono quelli richiedenti bassa latenza e basso bit-rate.

Il dimensionamento effettivo dei servizi a disposizione ed il loro numero varia in base alla configurazione prevista, ovvero:

- Caratteristiche del modulo radio utilizzato.
- Degrado del canale atteso.
- Configurazione del software.

In contesti dinamici, un dimensionamento conservativo della banda assicura stabilità anche in condizioni di forte attenuazione del segnale.

Le evidenze raccolte confermano l'efficacia del sistema per servizi di telemetria, interfono e visuale di bordo menzionati in sezione 6.2.2, se opportunamente dimensionati. Parallelamente, per dispositivi radiocomandati, si ottiene un controllo a bassa latenza con canali virtuali in numero maggiore rispetto alle soluzioni convenzionali del modellismo. Tali applicazioni evidenziano la versatilità della soluzione verso futuri sviluppi, quali la telepresenza a basso costo.

Glossario

- approccio naïf** Metodo o atteggiamento diretto, semplice e privo di preconcetti, spesso caratterizzato da una visione ingenua o sincera. ix
- best-effort** Soluzione senza garanzie, spesso con l'obiettivo di "funzionare e basta".. 2, 3
- cache miss** Si verifica quando i dati richiesti da un'applicazione non sono presenti nella memoria cache del processore.. 18, 21
- comunicazione Punto-Punto** Sessione fra due *endpoints*.
Non confondere con *Comunicazione End-to-End*, implica crittografia.
- duplex** Si dice **Comunicazione Full-Duplex** se l'invio e la ricezione dei dati sono simultanei, altrimenti **Comunicazione Half-Duplex**. 4, 28
- endpoint** Entità che invia e riceve dati.
- fat struct** Una struttura con molti elementi al suo interno.. 22
- frame** Un singolo messaggio. Può essere diviso in *segmenti*. vii, 5, 7, 9, 11, 12, 13, 14, 15, 16
- goodput** Velocità di scambio dati utile, senza overhead.. 32
- graceful reload** Si intende un riavvio del sistema senza perderne lo stato.. 23
- header e trailer** Dati aggiunti in testa o in coda a ogni segmento. Utili per correzione degli errori e interleaving. 9, 12, 14, 15, 17
- MTU** Maximum Transmission Unit : Unità Massima di Trasmissione.. 9, 10, 13
- ping** Tempo di andata e ritorno di un pacchetto.. 32
- protocollo di comunicazione** Insieme di regole che governa lo scambio. 4, 14
- race condition** Situazione in cui più thread lavorano contemporaneamente sulla stessa porzione di memoria.. 18, 19

Sitografia

- [1] Andrea Roccaccino. *rf24tunlink: Point to Point TUN link via nRF24L01 radio modules (English)*. URL: <https://github.com/LollosoSì/rf24tunlink> (visitato il giorno 18/01/2026).
- [2] Nordic Semiconductor. *nRF24L01: Single Chip 2.4GHz Transceiver (English)*. URL: https://cdn.sparkfun.com/datasheets/Wireless/Nordic/nRF24L01_Product_Specification_v2_0.pdf (visitato il giorno 12/12/2025).
- [3] Wikipedia. *TUN/TAP (English)*. URL: <https://en.wikipedia.org/wiki/TUN/TAP> (visitato il giorno 18/01/2026).
- [4] Wikipedia. *FEC (English)*. URL: https://en.wikipedia.org/wiki/Error_correction_code (visitato il giorno 18/01/2026).
- [5] StackOverflow. *What is move semantics? (English)*. URL: <https://stackoverflow.com/questions/3106110/what-is-move-semantics> (visitato il giorno 18/01/2026).
- [6] Wikipedia. *Reed Solomon Codes (English)*. URL: https://en.wikipedia.org/wiki/Reed%E2%80%93Solomon_error_correction (visitato il giorno 18/01/2026).
- [7] CPPReference. *std::atomic (English)*. URL: <https://en.cppreference.com/w/cpp/atomic/atomic.html> (visitato il giorno 18/01/2026).
- [8] CPPReference. *std::mutex (English)*. URL: <https://en.cppreference.com/w/cpp/thread/mutex.html> (visitato il giorno 18/01/2026).
- [9] CPPReference. *std::unique_lock (English)*. URL: https://en.cppreference.com/w/cpp/thread/unique_lock.html (visitato il giorno 18/01/2026).
- [10] CPPReference. *std::condition_variable (English)*. URL: https://en.cppreference.com/w/cpp/thread/condition_variable.html (visitato il giorno 18/01/2026).
- [11] TMRh20 e 2bndy5. *RF24: OSI Layer 2 driver for nRF24L01 on Arduino & Raspberry Pi/Linux Devices (English)*. URL: <https://github.com/nRF24/RF24> (visitato il giorno 18/01/2026).